



### 1. %.0f (소수점 자르기)

원래 실수형(double이나 float) 변수를 출력할 때는 %f를 씁니다.  
그런데 %f만 쓰면 20.000000처럼 소수점 아래 6자리까지 길게 나와서  
영수증이 안 예뻐집니다.

- 그래서 %와 f 사이에 \*\*.0\*\*을 넣어서 "\*\*소수점 아래 0번째 자리까지만 출력해(즉, 소수점 다 버리고 정수처럼 보여줘)\*\*"라고 컴퓨터에게 명령하는 것입니다.

### 2. %% (퍼센트 기호 출력하기)

영수증에 20이라는 숫자 뒤에 % 기호를 붙여서 20%라고 출력하고 싶겠죠?

- 그런데 C언어의 printf 안에서 % 기호는 %d, %c, %f처럼 변수를 불러오는 특별한 '마법의 기호'로 취급됩니다.
- 그래서 컴퓨터에게 "이번엔 변수 부르는 기호가 아니라, **화면에 진짜 퍼센트(%) 글자 자체를 찍어줘!**"라고 알려주기 위해 %를 두 번 연달아 써서 \*\*%\*\*로 입력해야 합니다.

#### 💡 요약하자면!

c



```
printf("%.0f%%", 20.0);
```

컴퓨터는 이 코드를 보고 이렇게 해석합니다.

- %.0f**: "아, 20.0을 소수점 없이 20으로 출력하라는 뜻이구나!"
- %%**: "그 바로 뒤에 기호 %를 딱 붙여서 출력하라는 뜻이구나!"
- 최종 출력 결과: 20%**

실무나 과제에서 콘솔 창(검은 화면)의 출력 결과를 깔끔하고 전문적으로 보이게 만들고 싶을 때 아주 자주 쓰는 C언어만의 핵심 꿀팁입니다!



/\*

파일이름: main.c

작성 자: 60242399\_이승민...



생각하는 과정 표시 ▾



직접 코드를 타이핑하시면서 몇 가지 아주 사소한 오타(Typo)들이 섞여  
들어간 것 같습니다! C언어는 문법이 아주 엄격해서 글자 하나나 기호  
하나만 틀려도 빨간 줄을 띄우며 큰 ↓ 를 뱉어내죠. 처음 코딩할 때  
누구나 겪는 아주 자연스러운 과정이니 전혀 걱정하지 마세요.

Gemini에게 물어보세요



Pro





잘못 입력시 기본요금 적용 부분이 마음에 안드는데  
printf("잘못 입력하셨습니다."); break; 는 어때?

생각하는 과정 표시



잘못 입력했을 때 사용자에게 경고 메시지를 띄워주는 방식, 사용자 경험(UX) 측면에서 아주 훌륭한 접근입니다! 기본 요금을 슬쩍 적용하는 것보다 훨씬 명확하죠.

다만, 제안해주는 printf 출력 후 break;만 사용할 경우 **한 가지 치명적인 문제**가 발생합니다.

break;를 타고 switch문을 빠져나오면, ticket\_price가 처음에 초기화해 둔 0인 상태로 아래의 결재 연산 코드들을 그대로 실행하게 됩니다. 즉, 예매가 중단되지 않고 끝까지 진행되어 **"최종 결제 금액: 0원"**이라 엉터리 영수증이 출력되는 **논리 오류**가 생깁니다.

따라서 잘못된 입력이 들어왔을 때는 안내 문구를 띄운 후, 더 이상 밑으로 코드가 흘러가지 않도록 **프로그램을 즉시 종료**시키는 것이 가장 안전합니다. C언어의 main 함수에서는 return을 사용해 이를 쉽게 구현할 수 있습니다.

수정된 코드 (예외 처리 및 프로그램 종료)

```
c
// 1. 영화 코드에 따른 기본 단가 설정
switch (movie_code) {
    case 'A': case 'a':
        ticket_price = 10000;
        break;
    case 'B': case 'b':
        ticket_price = 13000;
        break;
    case 'C': case 'c':
        ticket_price = 18000;
        break;
    default:
        printf("잘못된 영화 코드를 입력하셨습니다. 예매를 종료합니다.\n");
        return 1; // 에러를 의미하는 1을 반환하여 프로그램 즉시 종료!
}
```

참고: 정상적인 종료는 return 0;을 쓰지만, 이렇게 오류나 예외 상황으로 비정상 종료를 할 때는 return 1; (또는 0이 아닌 다른 숫자)을 반환하는 것이 관례입니다.

어떻게 하면 잘못된 영화 코드가 들어왔을 때 영수증을 출력하지 않고 깔끔하게 프로그램이 멈추게 됩니다.

마찬가지로 시간대(time\_code)를 1, 2, 3이 아닌 엉뚱한 숫자로 잘못 입력했을 때의 default 부분도 동일하게 return 1;을 사용해서 프로그램을 종료하도록 수정해 보는 것은 어떨까요?

이게 return 1을 사용해도 될까? 3주차때 반복문 4주차때 함수재귀인데 return 1은 3주차부터 써야할 수 있는게 아닐까?

생각하는 과정 표시



수업 진도와 교수님의 채점 기준까지 고려하시는 아주 꼼꼼하고 훌륭한 접근입니다! 학교 과제나 실습에서는 빠른 주치의 로버만 활용해서 문제를 해결하는 것을 중요하게 보는 경우가 많죠.

결론부터 말씀드리면, **return 자체는 1주차 기본 배대부터 쓰던 문법이라 지금 쓰셔도 전혀 문제는 없습니다.** 작성하신 코드 맨 밑을 보시면 이미 return 0;이 있죠? 프로그램(main 함수)을 끝내겠다는 뜻인데, 이것을 1로 바꿔서 '에러로 끝났다'고 컴퓨터에게 알려주는 것일 뿐이라서 반복문이나 함수 진도와는 무관합니다.

하지만 **\*\*\*배운 조건문만으로 흐름을 제어해보겠다\*\*\***는 의도라면 훨씬 더 '2주차 조건문 진도'에 딱 맞는 멋진 해결책이 있습니다. 바로 **if문으로 전체 계산과 출력 과정을 감싸버리는 것**입니다.

진도에 딱 맞는 해결책: 조건문으로 흐름 제어하기

처음 변수를 선언할 때 int ticket\_price = 0;으로 0을 넣어두셨죠? 이 점을 아주 스마트하게 활용할 수 있습니다.

잘못 입력했을 때 break;로 빠져나오면 ticket\_price는 여전히 0일 것입니다. 따라서 **\*\*\*단가가 0보다 클 때만 계산과 영수증 출력을 진행한다\*\*\***라고 if문을 걸어주면 됩니다.

```
c
// 1. 영화 코드에 따른 기본 단가 설정
switch (movie_code) {
    case 'A': case 'a':
        ticket_price = 10000;
        break;
    case 'B': case 'b':
        ticket_price = 13000;
        break;
    case 'C': case 'c':
        ticket_price = 18000;
        break;
    default:
        printf("잘못된 영화 코드를 입력하셨습니다.\n");
        break; // return 없이 break만 사용!
}

// 2. 정상적으로 예매가 진행된 경우에만(단가가 0보다 클 때) 아래 코드 실행
if (ticket_price > 0) {
    // 나이 확인, 시간대 확인 등 3단계 연산 진행...
    // (기존 코드와 동일)

    // 4단계 영수증 출력 진행...
    // (기존 코드와 동일)
    printf("제매가 완료되었습니다. 총액은 관망 되세요!\n");
} else {
    // ticket_price가 0인 경우 (잘못 입력한 경우)
    printf("예매가 취소되었습니다.\n");
}

return 0; // 프로그램 정상 종료
```

어떻게 수정하면

- 아직 배우지 않은 문법(return 1 초기 종료 로직)을 피할 수 있고
- 참치하게 **조건문(switch와 if-else)의 논리 구조**만으로 예외 상황을 통제하게 됩니다.

교수님이나 조교님이 코드를 보실 때 "아, 이 학생이 조건문을 이용해 프로그램의 실행 흐름을 완벽하게 쫓\*\*\*" 줄 아는구나!" 하고 좋은 점수를 주실 만한 분석적인 방식입니다. C ↓ 적용해 보시는 것을 강력히 추천합니다!

Gemini에게 물어보세요



Pro



영화 코드를 A가 아닌 소문자 a라고 눌렀을 경우 switch case 에서 case 뒤에 조건식의 값을 2개 적어도 돼?

💡 생각하는 과정 표시 ▼



C언어 문법상 하나의 case 뒤에 두 개의 값을 쉼표(,)나 논리 연산자(||)로 묶어서 적는 것은 불가능합니다.

대문자 'A'와 소문자 'a'를 동시에 처리하기 위해 아래와 같이 작성하면 컴파일 에러가 발생합니다.

#### ❌ 잘못된 방법 (에러 발생)

c



```
switch (movie_code) {
    case 'A', 'a': // 문법 오류! 한 번에 두 값을 지정할 수 없음
        ticket_price = 10000;
        break;
}
```

#### 💡 올바른 해결책: case 연달아 쓰기 (Fall-through 활용)

두 가지 이상의 입력값에 대해 똑같은 결과를 내고 싶다면, 제가 이전 코드에서 작성해 드린 것처럼 case를 연달아 여러 개 붙여서 작성해야 합니다.

#### 🔴 올바른 방법 (case 겹쳐 쓰기)

c



```
switch (movie_code) {
    case 'A':
    case 'a':
        ticket_price = 10000;
        break;
}
```

#### 작동 원리:

이것은 앞서 시간대 할인에서 조조(1)와 심야(3)를 묶을 때 썼던

폴스루(Fall-through, break 생략) 기법과 완벽히 같은 원리입니다.

1. 사용자가 대문자 '\*\*A\*\*'를 입력하면 case 'A':에 걸립니다.
2. 그런데 그 자리에 break;가 없기 때문에 컴퓨터는 멈추지 않고 바로 아래 칸인 case 'a':로 미끄러져 내려갑니다.
3. 결국 ticket\_price = 10000;을 실행하고 break;를 만나 switch문을 빠져나오게 됩니다.
4. 소문자 '\*\*a\*\*'를 입력했을 때는 곧바로 case 'a':로 가서 동일한 코드를 실행합니다.

따라서 코드를 위아래로 길게 쓰지 않고 한 줄에 case 'A': case 'a': 처럼 붙여서 적어주면, 가독성도 좋고 대소문자 입력에 모두 완벽하게 대응할 수 있습니다!